10.2 Hash Tables

In this section, we introduce one of the most practical data structures for implementing a map, and the one that is used by Python's own implementation of the dict class. This structure is known as a ***hash table***.

Intuitively, a map M supports the abstraction of using keys as indices with a syntax such as M[k]. As a mental warm-up, consider a restricted setting in which a map with n items uses keys that are known to be integers in a range from 0 to $N-1$ for some $N \geq n$. In this case, we can represent the map using a ***lookup table*** of length N, as diagrammed in Figure 10.3.

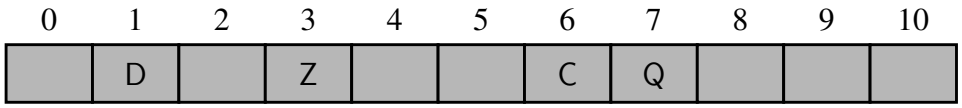


Figure 10.3: A lookup table with length 11 for a map containing items (1,D), (3,Z), (6,C), and (7,Q).

In this representation, we store the value associated with key k at index k of the table (presuming that we have a distinct way to represent an empty slot). Basic map operations of __getitem__, __setitem__, and __delitem__ can be implemented in $O(1)$ worst-case time.

There are two challenges in extending this framework to the more general setting of a map. First, we may not wish to devote an array of length N if it is the case that $N \gg n$. Second, we do not in general require that a map's keys be integers. The novel concept for a hash table is the use of a ***hash function*** to map general keys to corresponding indices in a table. Ideally, keys will be well distributed in the range from 0 to $N-1$ by a hash function, but in practice there may be two or more distinct keys that get mapped to the same index. As a result, we will conceptualize our table as a ***bucket array***, as shown in Figure 10.4, in which each bucket may manage a collection of items that are sent to a specific index by the hash function. (To save space, an empty bucket may be replaced by None.)

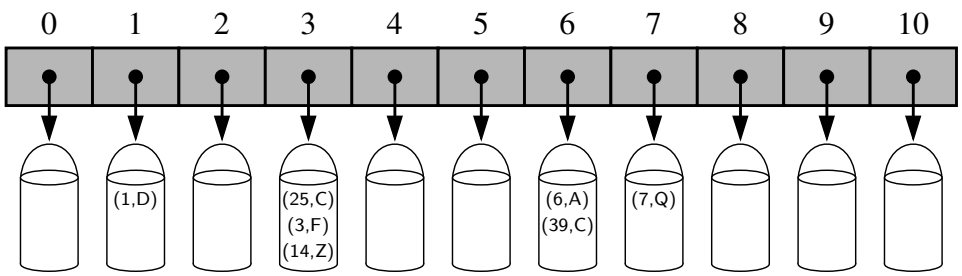


Figure 10.4: A bucket array of capacity 11 with items (1,D), (25,C), (3,F), (14,Z), (6,A), (39,C), and (7,Q), using a simple hash function.

10.2.1 Hash Functions

The goal of a ***hash function***, h, is to map each key k to an integer in the range $[0, N-1]$, where N is the capacity of the bucket array for a hash table. Equipped with such a hash function, h, the main idea of this approach is to use the hash function value, $h(k)$, as an index into our bucket array, A, instead of the key k (which may not be appropriate for direct use as an index). That is, we store the item (k, v) in the bucket $A[h(k)]$.

If there are two or more keys with the same hash value, then two different items will be mapped to the same bucket in A. In this case, we say that a ***collision*** has occurred. To be sure, there are ways of dealing with collisions, which we will discuss later, but the best strategy is to try to avoid them in the first place. We say that a hash function is "good" if it maps the keys in our map so as to sufficiently minimize collisions. For practical reasons, we also would like a hash function to be fast and easy to compute.

It is common to view the evaluation of a hash function, $h(k)$, as consisting of two portions—a ***hash code*** that maps a key k to an integer, and a ***compression function*** that maps the hash code to an integer within a range of indices, $[0, N-1]$, for a bucket array. (See Figure 10.5.)

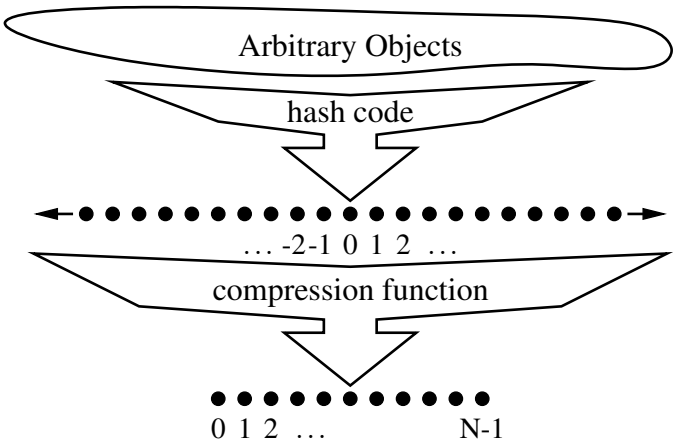


Figure 10.5: Two parts of a hash function: a hash code and a compression function.

The advantage of separating the hash function into two such components is that the hash code portion of that computation is independent of a specific hash table size. This allows the development of a general hash code for each object that can be used for a hash table of any size; only the compression function depends upon the table size. This is particularly convenient, because the underlying bucket array for a hash table may be dynamically resized, depending on the number of items currently stored in the map. (See Section 10.2.3.)

Hash Codes

The first action that a hash function performs is to take an arbitrary key k in our map and compute an integer that is called the ***hash code*** for k; this integer need not be in the range $[0, N-1]$, and may even be negative. We desire that the set of hash codes assigned to our keys should avoid collisions as much as possible. For if the hash codes of our keys cause collisions, then there is no hope for our compression function to avoid them. In this subsection, we begin by discussing the theory of hash codes. Following that, we discuss practical implementations of hash codes in Python.

Treating the Bit Representation as an Integer

To begin, we note that, for any data type X that is represented using at most as many bits as our integer hash codes, we can simply take as a hash code for X an integer interpretation of its bits. For example, the hash code for key 314 could simply be 314. The hash code for a floating-point number such as 3.14 could be based upon an interpretation of the bits of the floating-point representation as an integer.

For a type whose bit representation is longer than a desired hash code, the above scheme is not immediately applicable. For example, Python relies on 32-bit hash codes. If a floating-point number uses a 64-bit representation, its bits cannot be viewed directly as a hash code. One possibility is to use only the high-order 32 bits (or the low-order 32 bits). This hash code, of course, ignores half of the information present in the original key, and if many of the keys in our map only differ in these bits, then they will collide using this simple hash code.

A better approach is to combine in some way the high-order and low-order portions of a 64-bit key to form a 32-bit hash code, which takes all the original bits into consideration. A simple implementation is to add the two components as 32-bit numbers (ignoring overflow), or to take the exclusive-or of the two components. These approaches of combining components can be extended to any object x whose binary representation can be viewed as an n-tuple $(x_0, x_1, \ldots, x_{n-1})$ of 32-bit integers, for example, by forming a hash code for x as $\sum_{i=0}^{n-1} x_i$, or as $x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}$, where the $\oplus$ symbol represents the bitwise exclusive-or operation (which is ^ in Python).

Polynomial Hash Codes

The summation and exclusive-or hash codes, described above, are not good choices for character strings or other variable-length objects that can be viewed as tuples of the form $(x_0, x_1, \ldots, x_{n-1})$, where the order of the x_i's is significant. For example, consider a 16-bit hash code for a character string s that sums the Unicode values of the characters in s. This hash code unfortunately produces lots of unwanted

collisions for common groups of strings. In particular, "temp01" and "temp10" collide using this function, as do "stop", "tops", "pots", and "spot". A better hash code should somehow take into consideration the positions of the x_i's. An alternative hash code, which does exactly this, is to choose a nonzero constant, $a \neq 1$, and use as a hash code the value

$$x_0 a^{n-1} + x_1 a^{n-2} + \cdots + x_{n-2} a + x_{n-1}.$$

Mathematically speaking, this is simply a polynomial in a that takes the components $(x_0, x_1, \ldots, x_{n-1})$ of an object x as its coefficients. This hash code is therefore called a ***polynomial hash code***. By Horner's rule (see Exercise C-3.50), this polynomial can be computed as

$$x_{n-1} + a(x_{n-2} + a(x_{n-3} + \cdots + a(x_2 + a(x_1 + a x_0)) \cdots)).$$

Intuitively, a polynomial hash code uses multiplication by different powers as a way to spread out the influence of each component across the resulting hash code.

Of course, on a typical computer, evaluating a polynomial will be done using the finite bit representation for a hash code; hence, the value will periodically overflow the bits used for an integer. Since we are more interested in a good spread of the object x with respect to other keys, we simply ignore such overflows. Still, we should be mindful that such overflows are occurring and choose the constant a so that it has some nonzero, low-order bits, which will serve to preserve some of the information content even as we are in an overflow situation.

We have done some experimental studies that suggest that 33, 37, 39, and 41 are particularly good choices for a when working with character strings that are English words. In fact, in a list of over 50,000 English words formed as the union of the word lists provided in two variants of Unix, we found that taking a to be 33, 37, 39, or 41 produced less than 7 collisions in each case!

Cyclic-Shift Hash Codes

A variant of the polynomial hash code replaces multiplication by a with a cyclic shift of a partial sum by a certain number of bits. For example, a 5-bit cyclic shift of the 32-bit value 00111011001011010101000010101000 is achieved by taking the leftmost five bits and placing those on the rightmost side of the representation, resulting in 10110010110101010001010100000111. While this operation has little natural meaning in terms of arithmetic, it accomplishes the goal of varying the bits of the calculation. In Python, a cyclic shift of bits can be accomplished through careful use of the bitwise operators $<<$ and $>>$, taking care to truncate results to 32-bit integers.

An implementation of a cyclic-shift hash code computation for a character string in Python appears as follows:

```
def hash_code(s):
  mask = (1 << 32) - 1                # limit to 32-bit integers
  h = 0
  for character in s:
    h = (h << 5 & mask) | (h >> 27)   # 5-bit cyclic shift of running sum
    h += ord(character)               # add in value of next character
  return h
```

As with the traditional polynomial hash code, fine-tuning is required when using a cyclic-shift hash code, as we must wisely choose the amount to shift by for each new character. Our choice of a 5-bit shift is justified by experiments run on a list of just over 230,000 English words, comparing the number of collisions for various shift amounts (see Table 10.1).

	Collisions	
Shift	**Total**	**Max**
0	234735	623
1	165076	43
2	38471	13
3	7174	5
4	1379	3
5	190	3
6	502	2
7	560	2
8	5546	4
9	393	3
10	5194	5
11	11559	5
12	822	2
13	900	4
14	2001	4
15	19251	8
16	211781	37

Table 10.1: Comparison of collision behavior for the cyclic-shift hash code as applied to a list of 230,000 English words. The "Total" column records the total number of words that collide with at least one other, and the "Max" column records the maximum number of words colliding at any one hash code. Note that with a cyclic shift of 0, this hash code reverts to the one that simply sums all the characters.

Hash Codes in Python

The standard mechanism for computing hash codes in Python is a built-in function with signature hash(x) that returns an integer value that serves as the hash code for object x. However, only ***immutable*** data types are deemed hashable in Python. This restriction is meant to ensure that a particular object's hash code remains constant during that object's lifespan. This is an important property for an object's use as a key in a hash table. A problem could occur if a key were inserted into the hash table, yet a later search were performed for that key based on a different hash code than that which it had when inserted; the wrong bucket would be searched.

Among Python's built-in data types, the immutable int, float, str, tuple, and frozenset classes produce robust hash codes, via the hash function, using techniques similar to those discussed earlier in this section. Hash codes for character strings are well crafted based on a technique similar to polynomial hash codes, except using exclusive-or computations rather than additions. If we repeat the experiment described in Table 10.1 using Python's built-in hash codes, we find that only 8 strings out of the set of more than 230,000 collide with another. Hash codes for tuples are computed with a similar technique based upon a combination of the hash codes of the individual elements of the tuple. When hashing a frozenset, the order of the elements should be irrelevant, and so a natural option is to compute the exclusive-or of the individual hash codes without any shifting. If hash(x) is called for an instance x of a mutable type, such as a list, a TypeError is raised.

Instances of user-defined classes are treated as unhashable by default, with a TypeError raised by the hash function. However, a function that computes hash codes can be implemented in the form of a special method named __hash__ within a class. The returned hash code should reflect the immutable attributes of an instance. It is common to return a hash code that is itself based on the computed hash of the combination of such attributes. For example, a Color class that maintains three numeric red, green, and blue components might implement the method as:

```
def __hash__(self):
  return hash( (self._red, self._green, self._blue) )   # hash combined tuple
```

An important rule to obey is that if a class defines equivalence through __eq__, then any implementation of __hash__ must be consistent, in that if x == y, then hash(x) == hash(y). This is important because if two instances are considered to be equivalent and one is used as a key in a hash table, a search for the second instance should result in the discovery of the first. It is therefore important that the hash code for the second match the hash code for the first, so that the proper bucket is examined. This rule extends to any well-defined comparisons between objects of different classes. For example, since Python treats the expression 5 == 5.0 as true, it ensures that hash(5) and hash(5.0) are the same.

Compression Functions

The hash code for a key k will typically not be suitable for immediate use with a bucket array, because the integer hash code may be negative or may exceed the capacity of the bucket array. Thus, once we have determined an integer hash code for a key object k, there is still the issue of mapping that integer into the range $[0, N-1]$. This computation, known as a ***compression function***, is the second action performed as part of an overall hash function. A good compression function is one that minimizes the number of collisions for a given set of distinct hash codes.

The Division Method

A simple compression function is the ***division method***, which maps an integer i to

$$i \bmod N,$$

where N, the size of the bucket array, is a fixed positive integer. Additionally, if we take N to be a prime number, then this compression function helps "spread out" the distribution of hashed values. Indeed, if N is not prime, then there is greater risk that patterns in the distribution of hash codes will be repeated in the distribution of hash values, thereby causing collisions. For example, if we insert keys with hash codes $\{200, 205, 210, 215, 220, \ldots, 600\}$ into a bucket array of size 100, then each hash code will collide with three others. But if we use a bucket array of size 101, then there will be no collisions. If a hash function is chosen well, it should ensure that the probability of two different keys getting hashed to the same bucket is $1/N$. Choosing N to be a prime number is not always enough, however, for if there is a repeated pattern of hash codes of the form $pN + q$ for several different p's, then there will still be collisions.

The MAD Method

A more sophisticated compression function, which helps eliminate repeated patterns in a set of integer keys, is the ***Multiply-Add-and-Divide*** (or "MAD") method. This method maps an integer i to

$$[(ai + b) \bmod p] \bmod N,$$

where N is the size of the bucket array, p is a prime number larger than N, and a and b are integers chosen at random from the interval $[0, p-1]$, with $a > 0$. This compression function is chosen in order to eliminate repeated patterns in the set of hash codes and get us closer to having a "good" hash function, that is, one such that the probability any two different keys collide is $1/N$. This good behavior would be the same as we would have if these keys were "thrown" into A uniformly at random.

10.2.2 Collision-Handling Schemes

The main idea of a hash table is to take a bucket array, A, and a hash function, h, and use them to implement a map by storing each item (k, v) in the "bucket" $A[h(k)]$. This simple idea is challenged, however, when we have two distinct keys, k_1 and k_2, such that $h(k_1) = h(k_2)$. The existence of such ***collisions*** prevents us from simply inserting a new item (k, v) directly into the bucket $A[h(k)]$. It also complicates our procedure for performing insertion, search, and deletion operations.

Separate Chaining

A simple and efficient way for dealing with collisions is to have each bucket $A[j]$ store its own secondary container, holding items (k, v) such that $h(k) = j$. A natural choice for the secondary container is a small map instance implemented using a list, as described in Section 10.1.5. This ***collision resolution*** rule is known as ***separate chaining***, and is illustrated in Figure 10.6.

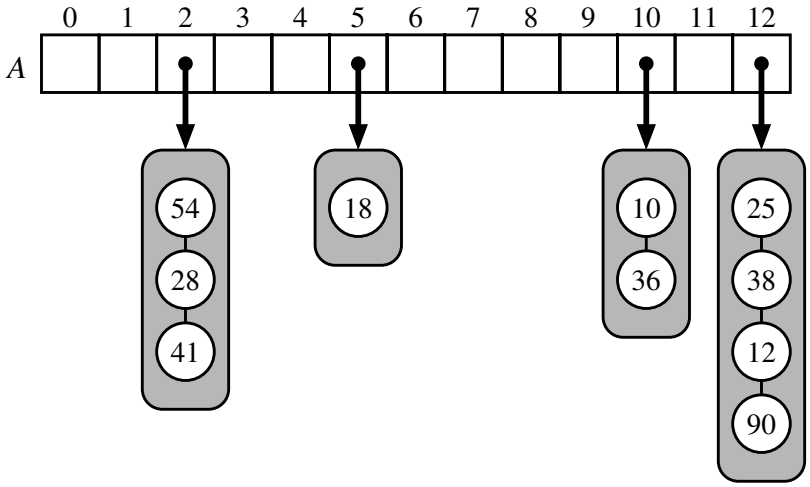


Figure 10.6: A hash table of size 13, storing 10 items with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

In the worst case, operations on an individual bucket take time proportional to the size of the bucket. Assuming we use a good hash function to index the n items of our map in a bucket array of capacity N, the expected size of a bucket is n/N. Therefore, if given a good hash function, the core map operations run in $O(\lceil n/N \rceil)$. The ratio $\lambda = n/N$, called the ***load factor*** of the hash table, should be bounded by a small constant, preferably below 1. As long as λ is $O(1)$, the core operations on the hash table run in $O(1)$ expected time.

Open Addressing

The separate chaining rule has many nice properties, such as affording simple implementations of map operations, but it nevertheless has one slight disadvantage: It requires the use of an auxiliary data structure—a list—to hold items with colliding keys. If space is at a premium (for example, if we are writing a program for a small handheld device), then we can use the alternative approach of always storing each item directly in a table slot. This approach saves space because no auxiliary structures are employed, but it requires a bit more complexity to deal with collisions. There are several variants of this approach, collectively referred to as ***open addressing*** schemes, which we discuss next. Open addressing requires that the load factor is always at most 1 and that items are stored directly in the cells of the bucket array itself.

Linear Probing and Its Variants

A simple method for collision handling with open addressing is ***linear probing***. With this approach, if we try to insert an item (k, v) into a bucket $A[j]$ that is already occupied, where $j = h(k)$, then we next try $A[(j+1) \bmod N]$. If $A[(j+1) \bmod N]$ is also occupied, then we try $A[(j+2) \bmod N]$, and so on, until we find an empty bucket that can accept the new item. Once this bucket is located, we simply insert the item there. Of course, this collision resolution strategy requires that we change the implementation when searching for an existing key—the first step of all `__getitem__`, `__setitem__`, or `__delitem__` operations. In particular, to attempt to locate an item with key equal to k, we must examine consecutive slots, starting from $A[h(k)]$, until we either find an item with that key or we find an empty bucket. (See Figure 10.7.) The name "linear probing" comes from the fact that accessing a cell of the bucket array can be viewed as a "probe."

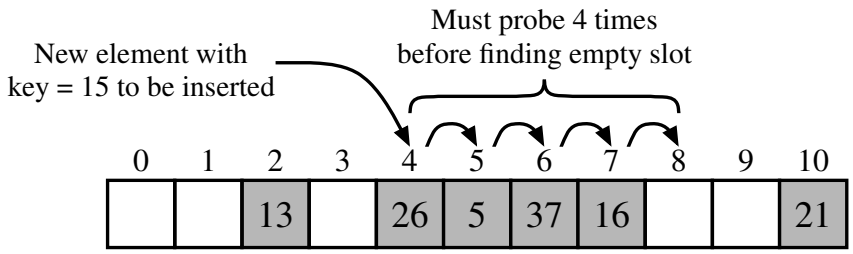


Figure 10.7: Insertion into a hash table with integer keys using linear probing. The hash function is $h(k) = k \bmod 11$. Values associated with keys are not shown.

To implement a deletion, we cannot simply remove a found item from its slot in the array. For example, after the insertion of key 15 portrayed in Figure 10.7, if the item with key 37 were trivially deleted, a subsequent search for 15 would fail because that search would start by probing at index 4, then index 5, and then index 6, at which an empty cell is found. A typical way to get around this difficulty is to replace a deleted item with a special "available" marker object. With this special marker possibly occupying spaces in our hash table, we modify our search algorithm so that the search for a key k will skip over cells containing the available marker and continue probing until reaching the desired item or an empty bucket (or returning back to where we started from). Additionally, our algorithm for `__setitem__` should remember an available cell encountered during the search for k, since this is a valid place to put a new item (k, v), if no existing item is found.

Although use of an open addressing scheme can save space, linear probing suffers from an additional disadvantage. It tends to cluster the items of a map into contiguous runs, which may even overlap (particularly if more than half of the cells in the hash table are occupied). Such contiguous runs of occupied hash cells cause searches to slow down considerably.

Another open addressing strategy, known as ***quadratic probing***, iteratively tries the buckets $A[(h(k) + f(i)) \bmod N]$, for $i = 0, 1, 2, \ldots$, where $f(i) = i^2$, until finding an empty bucket. As with linear probing, the quadratic probing strategy complicates the removal operation, but it does avoid the kinds of clustering patterns that occur with linear probing. Nevertheless, it creates its own kind of clustering, called ***secondary clustering***, where the set of filled array cells still has a non-uniform pattern, even if we assume that the original hash codes are distributed uniformly. When N is prime and the bucket array is less than half full, the quadratic probing strategy is guaranteed to find an empty slot. However, this guarantee is not valid once the table becomes at least half full, or if N is not chosen as a prime number; we explore the cause of this type of clustering in an exercise (C-10.36).

An open addressing strategy that does not cause clustering of the kind produced by linear probing or the kind produced by quadratic probing is the ***double hashing*** strategy. In this approach, we choose a secondary hash function, h', and if h maps some key k to a bucket $A[h(k)]$ that is already occupied, then we iteratively try the buckets $A[(h(k) + f(i)) \bmod N]$ next, for $i = 1, 2, 3, \ldots$, where $f(i) = i \cdot h'(k)$. In this scheme, the secondary hash function is not allowed to evaluate to zero; a common choice is $h'(k) = q - (k \bmod q)$, for some prime number $q < N$. Also, N should be a prime.

Another approach to avoid clustering with open addressing is to iteratively try buckets $A[(h(k) + f(i)) \bmod N]$ where $f(i)$ is based on a pseudo-random number generator, providing a repeatable, but somewhat arbitrary, sequence of subsequent probes that depends upon bits of the original hash code. This is the approach currently used by Python's dictionary class.

10.2.3 Load Factors, Rehashing, and Efficiency

In the hash table schemes described thus far, it is important that the load factor, $\lambda = n/N$, be kept below 1. With separate chaining, as λ gets very close to 1, the probability of a collision greatly increases, which adds overhead to our operations, since we must revert to linear-time list-based methods in buckets that have collisions. Experiments and average-case analyses suggest that we should maintain $\lambda < 0.9$ for hash tables with separate chaining.

With open addressing, on the other hand, as the load factor λ grows beyond 0.5 and starts approaching 1, clusters of entries in the bucket array start to grow as well. These clusters cause the probing strategies to "bounce around" the bucket array for a considerable amount of time before they find an empty slot. In Exercise C-10.36, we explore the degradation of quadratic probing when $\lambda \geq 0.5$. Experiments suggest that we should maintain $\lambda < 0.5$ for an open addressing scheme with linear probing, and perhaps only a bit higher for other open addressing schemes (for example, Python's implementation of open addressing enforces that $\lambda < 2/3$).

If an insertion causes the load factor of a hash table to go above the specified threshold, then it is common to resize the table (to regain the specified load factor) and to reinsert all objects into this new table. Although we need not define a new hash code for each object, we do need to reapply a new compression function that takes into consideration the size of the new table. Each ***rehashing*** will generally scatter the items throughout the new bucket array. When rehashing to a new table, it is a good requirement for the new array's size to be at least double the previous size. Indeed, if we always double the size of the table with each rehashing operation, then we can amortize the cost of rehashing all the entries in the table against the time used to insert them in the first place (as with dynamic arrays; see Section 5.3).

Efficiency of Hash Tables

Although the details of the average-case analysis of hashing are beyond the scope of this book, its probabilistic basis is quite intuitive. If our hash function is good, then we expect the entries to be uniformly distributed in the N cells of the bucket array. Thus, to store n entries, the expected number of keys in a bucket would be $\lceil n/N \rceil$, which is $O(1)$ if n is $O(N)$.

The costs associated with a periodic rehashing, to resize a table after occasional insertions or deletions can be accounted for separately, leading to an additional $O(1)$ amortized cost for `__setitem__` and `__getitem__`.

In the worst case, a poor hash function could map every item to the same bucket. This would result in linear-time performance for the core map operations with separate chaining, or with any open addressing model in which the secondary sequence of probes depends only on the hash code. A summary of these costs is given in Table 10.2.

Operation	List	Hash Table	
		expected	worst case
__getitem__	$O(n)$	$O(1)$	$O(n)$
__setitem__	$O(n)$	$O(1)$	$O(n)$
__delitem__	$O(n)$	$O(1)$	$O(n)$
__len__	$O(1)$	$O(1)$	$O(1)$
__iter__	$O(n)$	$O(n)$	$O(n)$

Table 10.2: Comparison of the running times of the methods of a map realized by means of an unsorted list (as in Section 10.1.5) or a hash table. We let n denote the number of items in the map, and we assume that the bucket array supporting the hash table is maintained such that its capacity is proportional to the number of items in the map.

In practice, hash tables are among the most efficient means for implementing a map, and it is essentially taken for granted by programmers that their core operations run in constant time. Python's dict class is implemented with hashing, and the Python interpreter relies on dictionaries to retrieve an object that is referenced by an identifier in a given namespace. (See Sections 1.10 and 2.5.) The basic command c = a + b involves two calls to __getitem__ in the dictionary for the local namespace to retrieve the values identified as a and b, and a call to __setitem__ to store the result associated with name c in that namespace. In our own algorithm analysis, we simply presume that such dictionary operations run in constant time, independent of the number of entries in the namespace. (Admittedly, the number of entries in a typical namespace can almost surely be bounded by a constant.)

In a 2003 academic paper [31], researchers discuss the possibility of exploiting a hash table's worst-case performance to cause a denial-of-service (DoS) attack of Internet technologies. For many published algorithms that compute hash codes, they note that an attacker could precompute a very large number of moderate-length strings that all hash to the identical 32-bit hash code. (Recall that by any of the hashing schemes we describe, other than double hashing, if two keys are mapped to the same hash code, they will be inseparable in the collision resolution.)

In late 2011, another team of researchers demonstrated an implementation of just such an attack [61]. Web servers allow a series of key-value parameters to be embedded in a URL using a syntax such as `?key1=val1&key2=val2&key3=val3`. Typically, those key-value pairs are immediately stored in a map by the server, and a limit is placed on the length and number of such parameters presuming that storage time in the map will be linear in the number of entries. If all keys were to collide, that storage requires quadratic time (causing the server to perform an inordinate amount of work). In spring of 2012, Python developers distributed a security patch that introduces randomization into the computation of hash codes for strings, making it less tractable to reverse engineer a set of colliding strings.

10.2.4 Python Hash Table Implementation

In this section, we develop two implementations of a hash table, one using separate chaining and the other using open addressing with linear probing. While these approaches to collision resolution are quite different, there are a great many commonalities to the hashing algorithms. For that reason, we extend the MapBase class (from Code Fragment 10.2), to define a new HashMapBase class (see Code Fragment 10.4), providing much of the common functionality to our two hash table implementations. The main design elements of the HashMapBase class are:

- The bucket array is represented as a Python list, named self._table, with all entries initialized to None.
- We maintain an instance variable self._n that represents the number of distinct items that are currently stored in the hash table.
- If the load factor of the table increases beyond 0.5, we double the size of the table and rehash all items into the new table.
- We define a _hash_function utility method that relies on Python's built-in hash function to produce hash codes for keys, and a randomized Multiply-Add-and-Divide (MAD) formula for the compression function.

What is not implemented in the base class is any notion of how a "bucket" should be represented. With separate chaining, each bucket will be an independent structure. With open addressing, however, there is no tangible container for each bucket; the "buckets" are effectively interleaved due to the probing sequences.

In our design, the HashMapBase class presumes the following to be abstract methods, which must be implemented by each concrete subclass:

- _bucket_getitem(j, k)
  This method should search bucket j for an item having key k, returning the associated value, if found, or else raising a KeyError.
- _bucket_setitem(j, k, v)
  This method should modify bucket j so that key k becomes associated with value v. If the key already exists, the new value overwrites the existing value. Otherwise, a new item is inserted and *this method is responsible for incrementing* self._n.
- _bucket_delitem(j, k)
  This method should remove the item from bucket j having key k, or raise a KeyError if no such item exists. (self._n is decremented *after* this method.)
- __iter__
  This is the standard map method to iterate through all keys of the map. Our base class does not delegate this on a per-bucket basis because "buckets" in open addressing are not inherently disjoint.

```
1  class HashMapBase(MapBase):
2    """Abstract base class for map using hash-table with MAD compression."""
3
4    def __init__(self, cap=11, p=109345121):
5      """Create an empty hash-table map."""
6      self._table = cap * [ None ]
7      self._n = 0                                 # number of entries in the map
8      self._prime = p                             # prime for MAD compression
9      self._scale = 1 + randrange(p-1)            # scale from 1 to p-1 for MAD
10     self._shift = randrange(p)                  # shift from 0 to p-1 for MAD
11
12   def _hash_function(self, k):
13     return (hash(k)*self._scale + self._shift) % self._prime % len(self._table)
14
15   def __len__(self):
16     return self._n
17
18   def __getitem__(self, k):
19     j = self._hash_function(k)
20     return self._bucket_getitem(j, k)           # may raise KeyError
21
22   def __setitem__(self, k, v):
23     j = self._hash_function(k)
24     self._bucket_setitem(j, k, v)               # subroutine maintains self._n
25     if self._n > len(self._table) // 2:         # keep load factor <= 0.5
26       self._resize(2 * len(self._table) - 1)    # number 2^x - 1 is often prime
27
28   def __delitem__(self, k):
29     j = self._hash_function(k)
30     self._bucket_delitem(j, k)                  # may raise KeyError
31     self._n -= 1
32
33   def _resize(self, c):                 # resize bucket array to capacity c
34     old = list(self.items())            # use iteration to record existing items
35     self._table = c * [None]            # then reset table to desired capacity
36     self._n = 0                         # n recomputed during subsequent adds
37     for (k,v) in old:
38       self[k] = v                       # reinsert old key-value pair
```

Code Fragment 10.4: A base class for our hash table implementations, extending our MapBase class from Code Fragment 10.2.

Separate Chaining

Code Fragment 10.5 provides a concrete implementation of a hash table with separate chaining, in the form of the ChainHashMap class. To represent a single bucket, it relies on an instance of the UnsortedTableMap class from Code Fragment 10.3.

The first three methods in the class use index j to access the potential bucket in the bucket array, and a check for the special case in which that table entry is None. The only time we need a new bucket structure is when _bucket_setitem is called on an otherwise empty slot. The remaining functionality relies on map behaviors that are already supported by the individual UnsortedTableMap instances. We need a bit of forethought to determine whether the application of __setitem__ on the chain causes a net increase in the size of the map (that is, whether the given key is new).

```
1  class ChainHashMap(HashMapBase):
2    """Hash map implemented with separate chaining for collision resolution."""
3
4    def _bucket_getitem(self, j, k):
5      bucket = self._table[j]
6      if bucket is None:
7        raise KeyError('Key Error: ' + repr(k))          # no match found
8      return bucket[k]                                   # may raise KeyError
9
10   def _bucket_setitem(self, j, k, v):
11     if self._table[j] is None:
12       self._table[j] = UnsortedTableMap( )             # bucket is new to the table
13     oldsize = len(self._table[j])
14     self._table[j][k] = v
15     if len(self._table[j]) > oldsize:                  # key was new to the table
16       self._n += 1                                     # increase overall map size
17
18   def _bucket_delitem(self, j, k):
19     bucket = self._table[j]
20     if bucket is None:
21       raise KeyError('Key Error: ' + repr(k))          # no match found
22     del bucket[k]                                      # may raise KeyError
23
24   def __iter__(self):
25     for bucket in self._table:
26       if bucket is not None:                           # a nonempty slot
27         for key in bucket:
28           yield key
```

Code Fragment 10.5: Concrete hash map class with separate chaining.

Linear Probing

Our implementation of a ProbeHashMap class, using open addressing with linear probing, is given in Code Fragments 10.6 and 10.7. In order to support deletions, we use a technique described in Section 10.2.2 in which we place a special marker in a table location at which an item has been deleted, so that we can distinguish between it and a location that has always been empty. In our implementation, we declare a class-level attribute, _AVAIL, as a sentinel. (We use an instance of the built-in object class because we do not care about any behaviors of the sentinel, just our ability to differentiate it from other objects.)

The most challenging aspect of open addressing is to properly trace the series of probes when collisions occur during an insertion or search for an item. To this end, we define a nonpublic utility, _find_slot, that searches for an item with key k in "bucket" j (that is, where j is the index returned by the hash function for key k).

```
1  class ProbeHashMap(HashMapBase):
2    """Hash map implemented with linear probing for collision resolution."""
3    _AVAIL = object( )       # sentinal marks locations of previous deletions
4
5    def _is_available(self, j):
6      """Return True if index j is available in table."""
7      return self._table[j] is None or self._table[j] is ProbeHashMap._AVAIL
8
9    def _find_slot(self, j, k):
10     """Search for key k in bucket at index j.
11
12     Return (success, index) tuple, described as follows:
13     If match was found, success is True and index denotes its location.
14     If no match found, success is False and index denotes first available slot.
15     """
16     firstAvail = None
17     while True:
18       if self._is_available(j):
19         if firstAvail is None:
20           firstAvail = j                          # mark this as first avail
21         if self._table[j] is None:
22           return (False, firstAvail)              # search has failed
23       elif k == self._table[j]._key:
24         return (True, j)                          # found a match
25       j = (j + 1) % len(self._table)              # keep looking (cyclically)
```

Code Fragment 10.6: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued in Code Fragment 10.7).

```
26   def _bucket_getitem(self, j, k):
27     found, s = self._find_slot(j, k)
28     if not found:
29       raise KeyError('Key Error: ' + repr(k))          # no match found
30     return self._table[s]._value
31
32   def _bucket_setitem(self, j, k, v):
33     found, s = self._find_slot(j, k)
34     if not found:
35       self._table[s] = self._Item(k,v)                 # insert new item
36       self._n += 1                                     # size has increased
37     else:
38       self._table[s]._value = v                        # overwrite existing
39
40   def _bucket_delitem(self, j, k):
41     found, s = self._find_slot(j, k)
42     if not found:
43       raise KeyError('Key Error: ' + repr(k))          # no match found
44     self._table[s] = ProbeHashMap._AVAIL               # mark as vacated
45
46   def __iter__(self):
47     for j in range(len(self._table)):                  # scan entire table
48       if not self._is_available(j):
49         yield self._table[j]._key
```

Code Fragment 10.7: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued from Code Fragment 10.6).

The three primary map operations each rely on the _find_slot utility. When attempting to retrieve the value associated with a given key, we must continue probing until we find the key, or until we reach a table slot with the None value. We cannot stop the search upon reaching an _AVAIL sentinel, because it represents a location that may have been filled when the desired item was once inserted.

When a key-value pair is being assigned in the map, we must attempt to find an existing item with the given key, so that we might overwrite its value, before adding a new item to the map. Therefore, we must search beyond any occurrences of the _AVAIL sentinel when inserting. However, if no match is found, we prefer to repurpose the first slot marked with _AVAIL, if any, when placing the new element in the table. The _find_slot method enacts this logic, continuing the search until a truly empty slot, but returning the index of the first available slot for an insertion.

When deleting an existing item within _bucket_delitem, we intentionally set the table entry to the _AVAIL sentinel in accordance with our strategy.